

NETWORK EVIDENCE ACQUISITION (PCAP), SHA-256 HASHING, AND BLOCKCHAIN SIMULATION

**Assignment: Network Evidence Acquisition, SHA-256 Hashing, and
Blockchain Simulation**



Student Identity

Name: Abdulaziz Bawadi

NIM: 23040700104

Course: Blockchain

UNIVERSITAS MUHAMMADIYAH JAKARTA

FAKULTAS TEKNIK

2026

1. Introduction

Digital evidence plays a critical role in digital forensics and network investigations. One of the main challenges in handling digital evidence is maintaining its integrity throughout the investigation process. Any modification to evidence may affect its validity and admissibility.

To address this issue, cryptographic hashing and blockchain technology can be used. Hashing provides a unique digital fingerprint for evidence, while blockchain ensures that evidence records remain tamper-resistant through chained blocks.

This project implements a simple blockchain system that stores metadata from PCAP network evidence files and validates their integrity using SHA-256 hashing.

2. Objectives

The objectives of this project are:

- To acquire network traffic evidence in PCAP format.
- To calculate SHA-256 hashes for each evidence file.
- To implement a simple blockchain for storing evidence metadata.
- To validate blockchain integrity.
- To demonstrate tampering detection through evidence modification.
- To understand the role of blockchain in maintaining chain of custody.

3. Network Evidence Acquisition

Network traffic evidence was generated and stored as PCAP files using Python and Scapy.

Five evidence files were created according to the assignment requirements:

File Name	Packet Count
PCAP01_23040700104.pcap	30
PCAP02_23040700104.pcap	50
PCAP03_23040700104.pcap	70
PCAP04_23040700104.pcap	90
PCAP05_23040700104.pcap	100

Each file contains a different number of packets to simulate different network traffic scenarios.

4. SHA-256 Hash Calculation

SHA-256 was used to generate a unique fingerprint for every PCAP evidence file.

The hashing process provides:

- Evidence authenticity verification.
- Tampering detection.
- Digital identification of evidence.

For each evidence file, the following information was collected:

- File name
- Packet count
- File size
- SHA-256 hash value

Because SHA-256 is highly sensitive to data changes, even a one-byte modification produces a completely different hash value.

```
PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23
ziz_Bawadi_23040700104\sourcecode\main.py"

Program Started Successfully

Generating required PCAP evidence files...

SHA-256 Evidence Results
-----
File Name   : PCAP01_23040700104.pcap
Packet Count : 30
File Size   : 3345 bytes
SHA-256     : 7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79
-----
File Name   : PCAP02_23040700104.pcap
Packet Count : 50
File Size   : 5565 bytes
SHA-256     : bfb9982ab0809d29fbc3ed29e9d0e2b50b4fb4d2f41344b109d7751854cfd64
-----
File Name   : PCAP03_23040700104.pcap
Packet Count : 70
File Size   : 7785 bytes
SHA-256     : 059070672dea54bd60109f7f560c744a09322210c6fb24ba60bb369b5b5e3a7c
-----
File Name   : PCAP04_23040700104.pcap
Packet Count : 90
File Size   : 10005 bytes
SHA-256     : 6702e7e62fbc93bec6e1319634acfed029f595d56c5761841e5c062f5c618723
-----
File Name   : PCAP05_23040700104.pcap
Packet Count : 100
File Size   : 11116 bytes
SHA-256     : 43668330748ef11a02a8f911c9ff95c8d7d43358a5a252e636a1254bab562db6
-----
```

5. Blockchain Structure

The blockchain consists of:

- One Genesis Block
- Five Evidence Blocks

Each block contains:

- Index
- Timestamp
- Evidence File Name
- Packet Count
- Evidence SHA-256 Hash
- Previous Hash
- Block Hash

The Previous Hash field links every block to the preceding block, creating an immutable chain.

Block Hash values are generated from all block attributes using SHA-256.

```
PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdul
ziz_Bawadi_23040700104\sourcecode\main.py"

Blockchain
-----
{
  "index": 0,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "GENESIS",
  "packet_count": 0,
  "evidence_sha256_hash": "0",
  "previous_hash": "0",
  "block_hash": "519f477109dba540cba40289739c42e56f63630a2a814133d0fae3394f5fcf3e"
}
-----
{
  "index": 1,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "PCAP01_23040700104.pcap",
  "packet_count": 30,
  "evidence_sha256_hash": "7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79",
  "previous_hash": "519f477109dba540cba40289739c42e56f63630a2a814133d0fae3394f5fcf3e",
  "block_hash": "66f5b50f6ad16d112bc854d4b1953fc9d96ac1c859973e63d4c3000679eb94c3"
}
-----
{
  "index": 2,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "PCAP02_23040700104.pcap",
  "packet_count": 50,
  "evidence_sha256_hash": "bfba9982ab0809d29fbc3ed29e9d0e2b50b4fb4d2f41344b109d7751854cfd64",
  "previous_hash": "66f5b50f6ad16d112bc854d4b1953fc9d96ac1c859973e63d4c3000679eb94c3",
  "block_hash": "1a3527494c24dfba6b97a5818ce0bfe2fcf29091a2b96ee91a18f8a76bc43be3"
}
-----
{
  "index": 3,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "PCAP03_23040700104.pcap",
  "packet_count": 70,
  "evidence_sha256_hash": "059070672dea54bd60109f7f560c744a09322210c6fb24ba60bb369b5b5e3a7c",
  "previous_hash": "1a3527494c24dfba6b97a5818ce0bfe2fcf29091a2b96ee91a18f8a76bc43be3",
  "block_hash": "73bd1b990ec289b1774a47aabfa85c7f4ff611488aa00e2f1c854274ab57bf00"
}
-----
```

```

{
  "index": 4,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "PCAP04_23040700104.pcap",
  "packet_count": 90,
  "evidence_sha256_hash": "6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723",
  "previous_hash": "73bd1b990ec289b1774a47aabfa85c7f4ff611488aa00e2f1c854274ab57bf00",
  "block_hash": "09e65805eb063ff0831f00cae6deeba4f8af397acf1978300b4a16cc8666edd2"
}
-----
{
  "index": 5,
  "timestamp": "2026-06-18T10:36:33+00:00",
  "evidence_file_name": "PCAP05_23040700104.pcap",
  "packet_count": 100,
  "evidence_sha256_hash": "43668330748ef11a02a8f911c9ff95c8d7d43358a5a252e636a1254bab562db6",
  "previous_hash": "09e65805eb063ff0831f00cae6deeba4f8af397acf1978300b4a16cc8666edd2",
  "block_hash": "68aacb9196b5cf3ff6fc7c5342fa95fa784168eabee730158fee834ed63ef344"
}
-----

```

6. Blockchain Validation Mechanism

The blockchain validation process verifies:

1. Previous Hash consistency.
2. Block Hash correctness.
3. Evidence Hash integrity.

The validation algorithm recalculates all hashes and compares them against stored values.

If all values match:

Blockchain Validation = VALID

If any mismatch is found:

Blockchain Validation = INVALID

This mechanism ensures that any modification to evidence can be immediately detected.

```

-----
Blockchain Validation: VALID

```

7. Merkle Tree Implementation

A Merkle Tree was implemented using the SHA-256 hashes of all evidence files.

Process:

1. SHA-256 hashes are used as leaf nodes.
2. Adjacent hashes are combined.

3. New parent hashes are calculated.
4. The process repeats until a single hash remains.

The final hash is called the Merkle Root.

Benefits:

- Efficient integrity verification.
- Reduced verification complexity.
- Commonly used in modern blockchain systems.

```
Merkle Tree
-----
Leaf Hashes
1. 7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79
2. bfba9982ab0809d29fbc3ed29e9d0e2b50b4fb4d2f41344b109d7751854cfd64
3. 059070672dea54bd60109f7f560c744a09322210c6fb24ba60bb369b5b5e3a7c
4. 6702e7e62fbc93bec6e1319634acfed029f595d56c5761841e5c062f5c618723
5. 43668330748ef11a02a8f911c9ff95c8d7d43358a5a252e636a1254bab562db6
Parent Level 1
1. 2c7cce1d73d52dc42e90c6908324b04f2f2144b9cf41534dfe58943ddca8f5b0
2. 068a8e8e80630fe8fb166e9b22effba9d89abaecb7ff221fc36f7a4dd6781c38
3. 1e4d82279b25775e1d55f9bc4f5a78c0ec9e59259564f38446ce47707a4f891d
Parent Level 2
1. 3918d30400d22d54ff0d9aef6fb3e6e8f9070f847455fd12b1a26d38ec29bb05
2. d5f500c9681242768841c52825fcf972697b9cfe73a1262ac1ed464b9999a6c6
Parent Level 3
1. 77aa290dad1441cb59e259f263b7125f3814d105e5e4de7796aef71e976839bc
Merkle Root: 77aa290dad1441cb59e259f263b7125f3814d105e5e4de7796aef71e976839bc
```

8. Tampering Simulation

A manual tampering test was performed.

Procedure:

1. Blockchain was created and validated.
2. One PCAP evidence file was manually modified.
3. SHA-256 hash was recalculated.
4. Blockchain validation was executed again.

Result:

Original evidence hash and modified evidence hash became different.

The blockchain successfully detected the modification and changed the validation status from VALID to INVALID.

This demonstrates the effectiveness of cryptographic hashing and blockchain-based integrity verification.

```
=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 1

Blockchain Validation: VALID
```

9. Results and Discussion

Before tampering:

- Blockchain Status: VALID
- All evidence hashes matched.
- All block hashes matched.
- Previous Hash links were correct.

After tampering:

- Evidence hash changed.
- Blockchain validation failed.
- Tampering was detected successfully.

The experiment demonstrates that blockchain can effectively preserve digital evidence integrity and support forensic investigations.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik
ziz_Bawadi_23040700104\sourcecode/main.py"

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 2

Available Files:
PCAP01_23040700104.pcap
PCAP02_23040700104.pcap
PCAP03_23040700104.pcap
PCAP04_23040700104.pcap
PCAP05_23040700104.pcap

Enter file name you modified manually: PCAP01_23040700104.pcap

Tampering Detected!

Old Hash:
7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79

New Hash:
7ea0dcf042a72f1b60340a0a1b08a5cfda311419e8c39c3d6493b0b6649b3e03

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 2

Available Files:
PCAP01_23040700104.pcap
PCAP02_23040700104.pcap
PCAP03_23040700104.pcap
PCAP04_23040700104.pcap
PCAP05_23040700104.pcap
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pa
ziz_Bawadi_23040700104/sourcecode/main.py"
Enter file name you modified manually: PCAP01_23040700104.pcap

No Tampering Detected.

Old Hash:
7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79

New Hash:
7714a55dd8808b8d1d4326e185c4622234180525638b6735f3de396982618f79

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 2

Available Files:
PCAP01_23040700104.pcap
PCAP02_23040700104.pcap
PCAP03_23040700104.pcap
PCAP04_23040700104.pcap
PCAP05_23040700104.pcap

Enter file name you modified manually: PCAP04_23040700104.pcap

Tampering Detected!

Old Hash:
6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723

New Hash:
63f31fd56ca7f3ab4937b5a56955718ae4368b49045df03bf9cab8453e956058

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 3
```

```
PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23040700104> & c:\python313\python.exe "c:\ziz_Bawadi_23040700104\sourcecode\main.py"

Merkle Root:
77aa290dad1441cb59e259f263b7125f3814d105e5e4de7796aef71e976839bc

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 4

Report generated:
C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23040700104\report\final_report.md

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 2

Available Files:
PCAP01_23040700104.pcap
PCAP02_23040700104.pcap
PCAP03_23040700104.pcap
PCAP04_23040700104.pcap
PCAP05_23040700104.pcap

Enter file name you modified manually: PCAP04_23040700104.pcap

No Tampering Detected.

Old Hash:
6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723

New Hash:
6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723
```

```
PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23040700104> & c:\python313\python

Enter file name you modified manually: PCAP04_23040700104.pcap

No Tampering Detected.

Old Hash:
6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723

New Hash:
6702e7e62fbe93bec6e1319634acfed029f595d56c5761841e5c062f5c618723

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 4

Report generated:
C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23040700104\report\final_report.md

=====
BLOCKCHAIN MENU
=====
1. Verify Blockchain
2. Tamper Evidence
3. Show Merkle Root
4. Generate Report
5. Exit

Choose option: 5

Exiting...
PS C:\Users\Administrator\Desktop\UMJ\SEMESTER 6\Block Chain\Tugas_Blockchain Pada Forensik Jaringan_Abdulaziz_Bawadi_23040700104> |
```

10. Conclusion

This project successfully implemented:

- Network evidence acquisition using PCAP files.
- SHA-256 hashing for digital evidence identification.
- A simple blockchain structure.
- Blockchain validation mechanisms.
- Merkle Tree construction.
- Manual tampering detection.

The results show that blockchain technology can significantly improve the integrity, traceability, and chain of custody of digital forensic evidence.

By combining SHA-256 hashing and blockchain, investigators can detect unauthorized modifications and maintain trustworthy evidence records throughout the forensic process.

References

Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.

Stallings, W. Cryptography and Network Security.

NIST Secure Hash Standard (SHA-256).

Python Documentation.

Scapy Documentation.

Video Presentation

<https://drive.google.com/file/d/1IWGD5dvYGGbTpKt2FediHWszZfyhH0bA/view?usp=sharing>